TED University
SENG 491 – Senior Project
Project Analysis Report: SpecOps
Fall 2025

Project Title: SpecOps: Specification-to-Code Pattern Enforcer

Course: SENG 491 – Senior Project

Team Members:

Shahdad Peykani – 99111760026 – Software Engineering

Muhammad Ozar Mirza – 99687965054 – Software Engineering

Abdirahman Tahlil Farah Hussein – 99717977422 – Software Engineering

Supervisor: Prof. Tansel Dökeroğlu

Jury Members:

Prof. Emin Kuğu

Prof. Elif Kurtaran Özbudak

Contents

Project Analysis Report: SpecOps 1
1. Executive Summary 3
2. Problem Definition 3
3. Proposed System 3
3.1 Overview 3
3.2 Functional Requirements 4
3.3 Nonfunctional Requirements 4
3.4 Pseudo Requirements 5
3.5 System Models 5
4. Glossary 9
5. References 9

1. Executive Summary

SpecOps is an AI-powered framework designed to enforce software engineering discipline upon Large Language Model (LLM) generated code. The system addresses the emerging problem of "vibe coding," where unstructured prompting leads to architectural debt and security vulnerabilities. By transforming natural language prompts into formal Software Requirements Specifications (SRS), selecting appropriate design patterns, and enforcing quality gates, SpecOps ensures that AI-assisted development remains structured, maintainable, and secure.

2. Problem Definition

The rapid adoption of LLMs in software development has introduced significant challenges:

- **"Vibe Coding":** Developers often use "just prompt and code" methodologies, resulting in chaotic prototyping without shared structure.
- **Architectural Debt:** Quickly generated code often lacks consistent design patterns, making future maintenance difficult.
- **Security Gaps:** Ad-hoc generation often overlooks standard security practices, requiring significant time for refactoring and cleaning.

3. Proposed System

3.1 Overview

SpecOps is a governance-focused software engineering framework designed to act as an autonomous "Lead Engineer." It utilizes a **Multi-Agent System** orchestrated via a central state machine.

The workflow enforces a strict pipeline:

1. **Interpretation:** The SpecParser agent converts natural language into a formal JSON-based SRS.
2. **Architecture:** The PatternSelector agent retrieves verified design patterns (via RAG) best suited for the requirements.
3. **Implementation:** The CodeGenerator agent produces code based on the selected pattern's Abstract Syntax Tree (AST) structure.
4. **Validation:** The QualityAuditor agent runs static analysis and security checks before the user sees the code.

3.2 Functional Requirements

These requirements expand on the initial SRS by adding technical implementation details required for the backend logic.

- FR-01: Structured SRS Extraction
  The system shall ingest a natural language prompt and output a standardized SRS in JSON format, validating fields (Scope, Functional Req, Non-Functional Req) against a strict schema to prevent LLM hallucination.
- FR-02: Pattern Recommendation & Justification
  The system shall analyze the SRS and propose at least three candidate design patterns. It must select the optimal one and generate a text-based justification citing trade-offs (e.g., "Selected Factory Pattern over Builder because object creation complexity is low").
- FR-03: AST-Based Code Generation
  The system shall generate source code that strictly adheres to the structural skeleton of the selected pattern.
- FR-04: Multi-Stage Validation Loop
  The system must autonomously execute a validation loop:
  - *Stage 1:* Syntax Check (Python compile()).
  - *Stage 2:* Static Analysis (Pylint score > 7/10).
  - Stage 3: Security Scan (Bandit - 0 High Severity issues).
    If Stage 1 or 2 fails, the system must feed the error log back to the Generator Agent for self-correction (max 3 retries).
- FR-05: Project Scaffolding
  The system shall generate a standardized directory structure (e.g., /src, /tests, /docs) and a requirements.txt file derived from imports found in the generated code.
- FR-06: Explainer Interface
  The system shall provide an interactive UI where users can hover over generated code blocks to see which SRS requirement necessitated that specific logic.

3.3 Nonfunctional Requirements

- NFR-01: Reliability (Hallucination Control)
  The system must utilize grounding techniques (RAG) to ensure generated design patterns match standard definitions (e.g., GoF definitions) with >95% structural accuracy.
- NFR-02: Performance (Latency)

The initial SRS generation must complete within 10 seconds. Full code generation and analysis for a medium-complexity project must complete within 180 seconds.

- NFR-03: Security (Sandbox)
  Generated code execution during the testing phase must occur in an isolated environment (e.g., Docker container or restricted subprocess) to prevent malicious code execution on the host server.
- NFR-04: Recoverability
  If an external API (Gemini) times out, the system must implement an exponential backoff retry strategy before reporting failure to the user.

3.4 Pseudo Requirements

Pseudo requirements describe the implementation constraints and technical standards imposed on the project.

- **PR-01: Technology Stack**
  - **Backend:** Python 3.10+ (Required for advanced AST matching features).
  - **API Framework:** FastAPI (for asynchronous task management).
  - **LLM Provider:** Gemini 2.5 Pro (via Google Generative AI SDK).
- PR-02: Data Interchange
  All inter-agent communication must be strictly typed using Pydantic Models to ensure the output of one agent (e.g., the SRS JSON) creates a valid input for the next (e.g., the Pattern Selector).
- PR-03: Compliance Standards
  Generated code must adhere to PEP 8 style guidelines. Security scanning must enforce OWASP Top 10 rules where applicable to Python.
- PR-04: Version Control Integration
  The system must be capable of initializing a local Git repository for the generated project.

3.5 System Models

3.5.1 Scenarios

Scenario A: The "Clean" Path (Success)

1. **User:** Enters prompt "Create a CLI To-Do list using a database."
2. **SpecOps:** Generates SRS identifying "Persistence" and "CLI Interface" as requirements.
3. **SpecOps:** Selects **Repository Pattern** (for database abstraction) and **Command Pattern** (for CLI actions).
4. **SpecOps:** Generates code. Pylint scores 9/10. Tests pass.

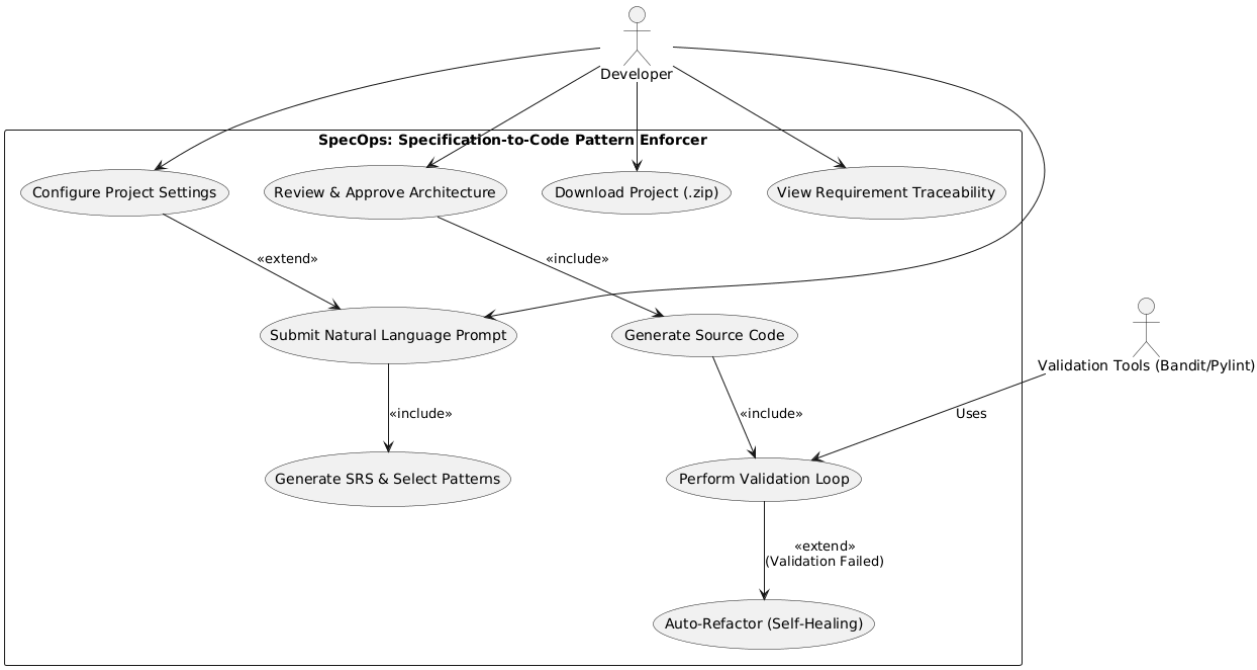
5. **Result:** User downloads the zip file.

Scenario B: The "Self-Healing" Path (Correction)

1. **User:** Enters prompt "Web scraper for news site."
2. **SpecOps:** Generates scraper code using eval() to parse data.
3. **Auditor:** Runs Bandit scan. Detects B307: eval used. Flags as High Severity.
4. **System:** Triggers "Fix Loop." Feeds error back to Code Generator with instruction: "Replace eval with json.loads."
5. **SpecOps:** Regenerates code. Rescans. Audit passes.
6. **Result:** Secure code delivered.

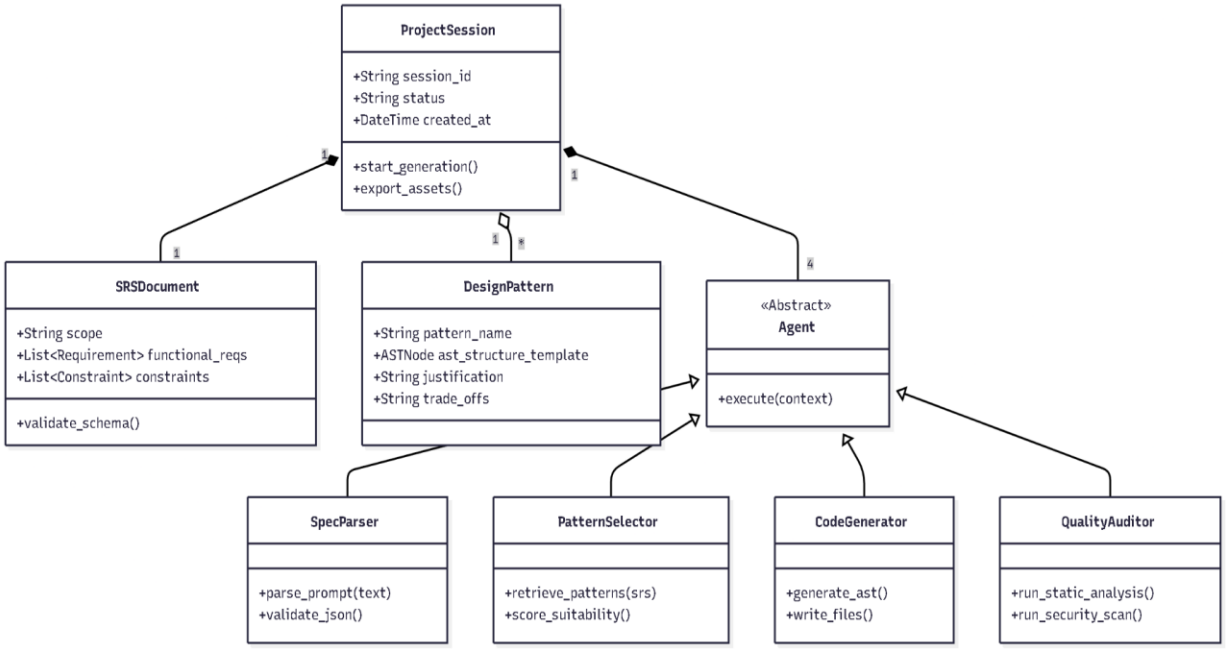
3.5.2 Use Case Model

The system interacts primarily with the **Developer** and external **Validation Tools**. The diagram below shows the five core functionalities available to the actors.



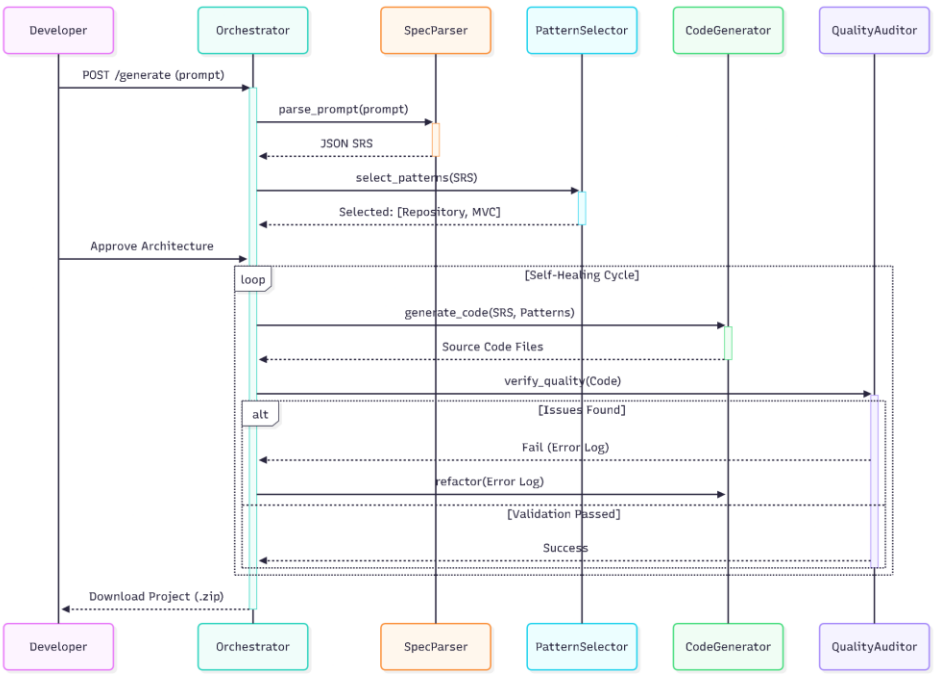
3.5.3 Object and Class Model

This model represents the internal data structures used by the Python backend. It details the *ProjectSession* management, the structure of the *SRS Document*, and the inheritance hierarchy of the AI *Agents*.



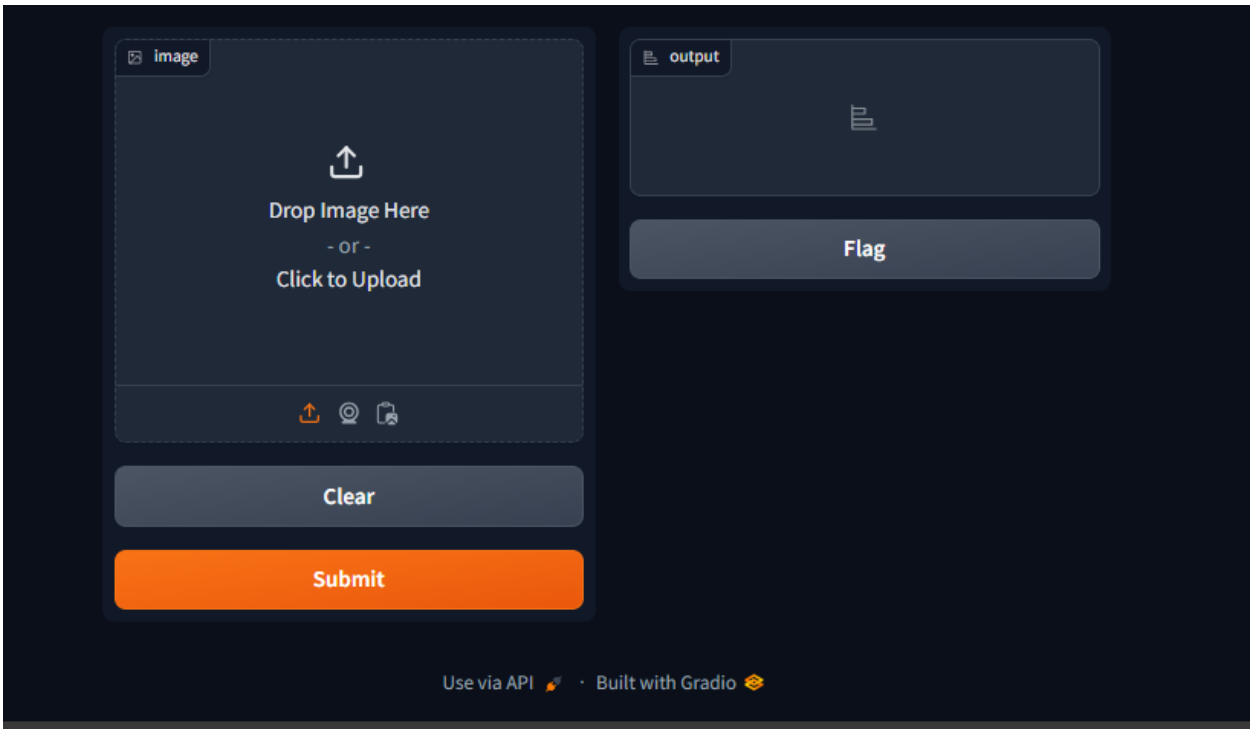
3.5.4 Dynamic Models

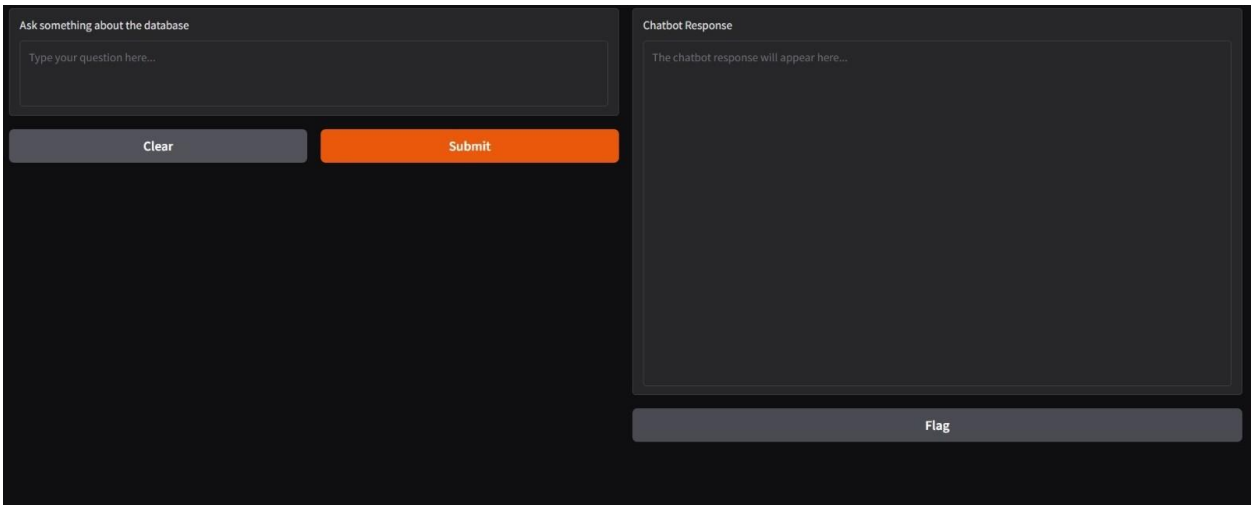
The sequence diagram below illustrates the time-ordered flow of the general pipeline. It specifically highlights the self-healing loop where the *QualityAuditor* rejects the code and forces the *CodeGenerator* to refactor it.



3.5.5 User Interface/Navigational Paths and Screen Mock-ups

The SpecOps interface utilizes a Gradio-based web layout to allow developers to submit natural language prompts and initiate the code generation pipeline. It serves as the primary interaction point for configuring project settings and reviewing the resulting software specifications and validation outputs.





4. Glossary

- **AST (Abstract Syntax Tree):** A tree representation of the abstract syntactic structure of source code. SpecOps uses this to verify if a "Singleton" pattern actually contains a private static instance.
- **Hallucination:** A phenomenon where an LLM generates plausible-sounding but factually incorrect or nonsensical information.
- **RAG (Retrieval-Augmented Generation):** A technique to optimize LLM output by referencing an authoritative knowledge base (in this case, a library of verified design patterns) before generating a response.
- **Vibe Coding:** A slang term for unstructured, prompt-driven programming where code is generated based on "feeling" or loose description rather than engineering specifications.
- **Grounding:** The process of linking abstract AI generations to concrete, verifiable constraints (like a JSON schema or a linter output).

5. References

1. Gamma, E., Helm, R., Johnson, R., & Vlissides, J. (1994). *Design Patterns: Elements of Reusable Object-Oriented Software*. Addison-Wesley.
2. OWASP Foundation. (2023). *OWASP Top Ten Security Risks*. https://owasp.org/www-project-top-ten/
3. Python Software Foundation. (n.d.). *ast — Abstract Syntax Tree*. Python 3.10 Documentation.
4. Google. (2024). *Gemini API Documentation*. https://ai.google.dev/
5. Richardson, L. (2013). *RESTful Web APIs*. O'Reilly Media.
6. MermaidChart. (n.d.). Mermaid Chart Online Editor. Retrieved from https://www.mermaidchart.com/app